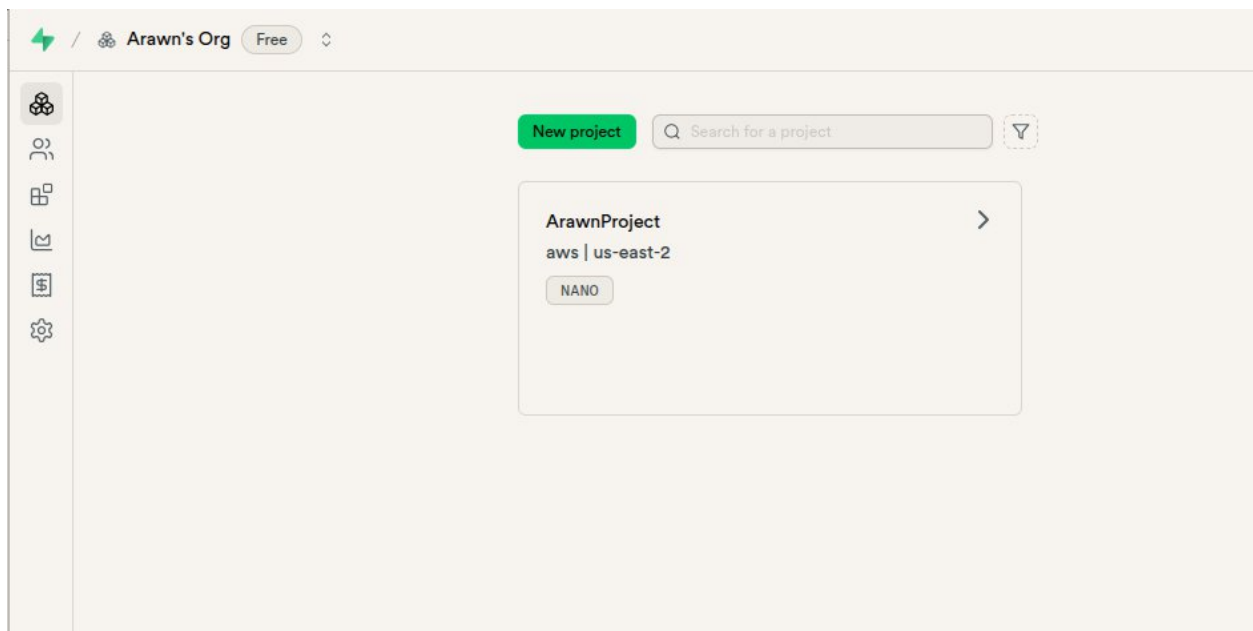


Crystal Save – Using Supabase as Cloud Saving Storage

Preparations:

1. Create a new Project



2. Enter your new Project

The screenshot shows the Supabase dashboard for a new project named 'ArawnProject'. The top navigation bar includes 'Arawn's Org', 'Free', 'ArawnProject', 'Connect', and 'Enable branching'. The main header displays 'ArawnProject' with a 'NANO' badge. On the right, there are statistics for 'Tables' (0), 'Functions' (0), and 'Replicas' (0), along with a 'Project Status' indicator showing a green dot and the text 'Project Status'.

The main content area is titled 'Welcome to your new project' and states: 'Your project has been deployed on its own instance, with its own API all set up and ready to use.'

Below this, there is a section titled 'Get started by building out your database'. It includes a description: 'Start building your app by creating tables and inserting data. Our Table Editor makes Postgres as easy to use as a spreadsheet, but there's also our SQL Editor if you need something more.' Below the description are three buttons: 'Table Editor', 'SQL Editor', and 'About Database'.

To the right of the 'Get started' section, there is a code editor showing SQL code for creating a 'todos' table and a table with 8 rows of data. The table has columns 'id', 'task', and 'status'.

Below the 'Get started' section, there is a section titled 'Explore our other products'. It includes a description: 'Supabase provides all the backend features you need to build a product. You can use it completely, or just the features you need.' Below the description are four product cards: 'Authentication', 'Storage', 'Edge Functions', and 'Realtime'. Each card has a description and two buttons: 'Explore' and 'About'.

id	task	status
1	Create a project	Complete
2	Read documentation	Complete
3	Build application	In progress
4	Connect Supabase	In progress
5	Deploy project	Not started
6	Get users	Not started
7		
8		

3. Click on 'Explore Storage' at bottom inside the box named 'Storage'

Explore our other products

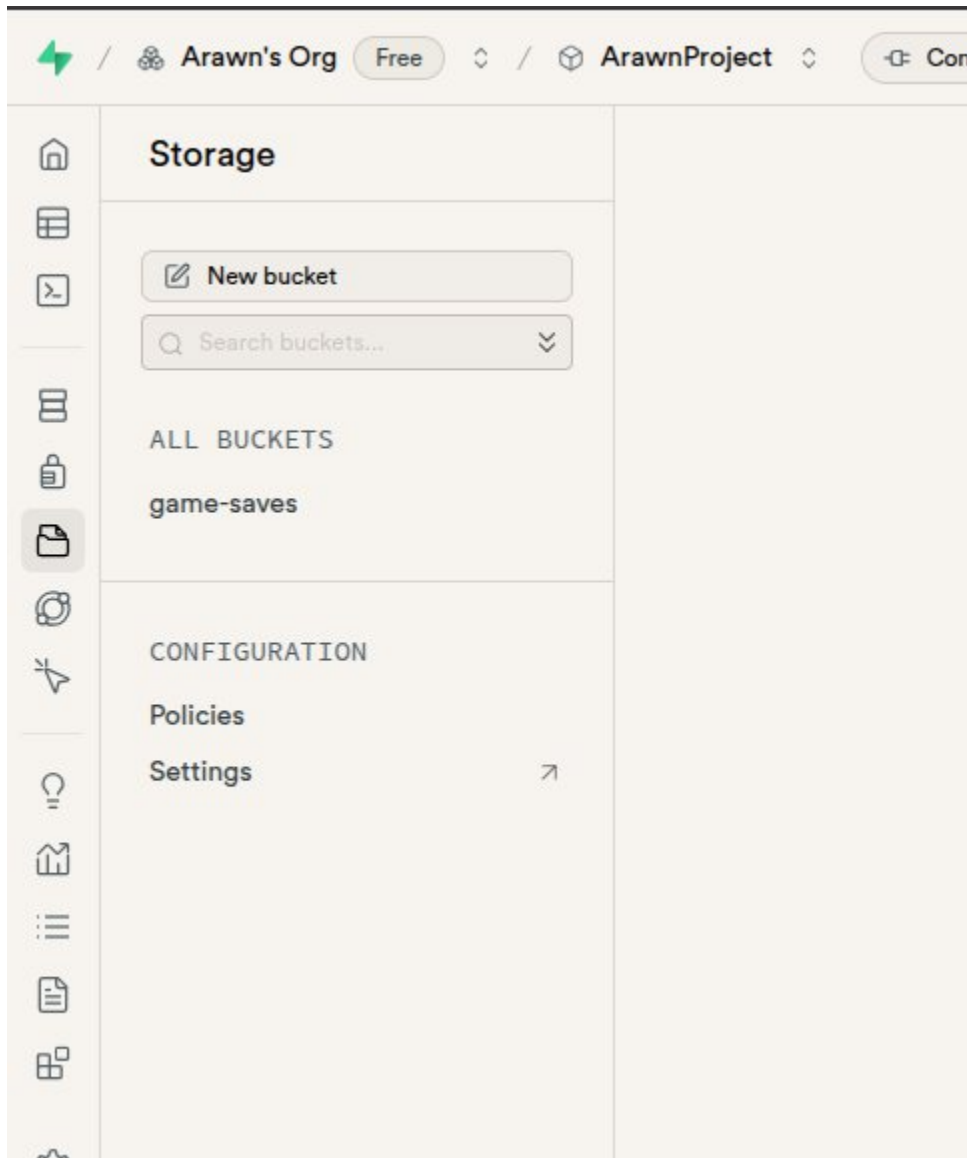
Supabase provides all the backend features you need to build a product. You can use it completely, or just the features you need.

This section shows a close-up of the 'Explore our other products' area. It features three product cards: 'Authentication', 'Storage', and 'Edge Functions'. Each card has a description and two buttons: 'Explore' and 'About'.

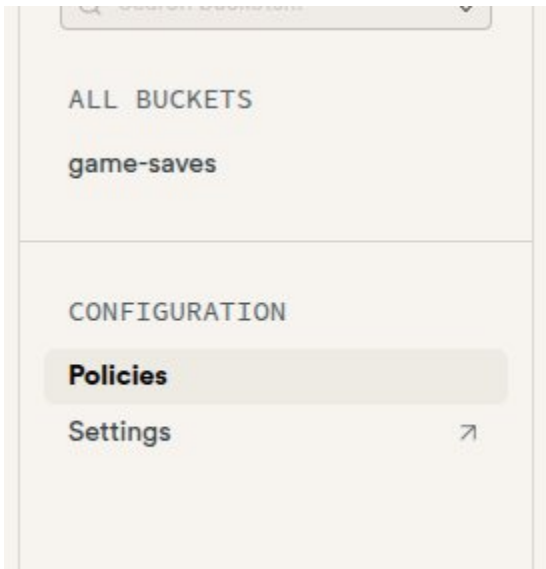
The 'Storage' card is highlighted, showing its description: 'Store, organize, and serve any file types of any size from multiple buckets.' Below the description are two buttons: 'Explore Storage' and 'About Storage'.

4. Create a new bucket, we name it for example:

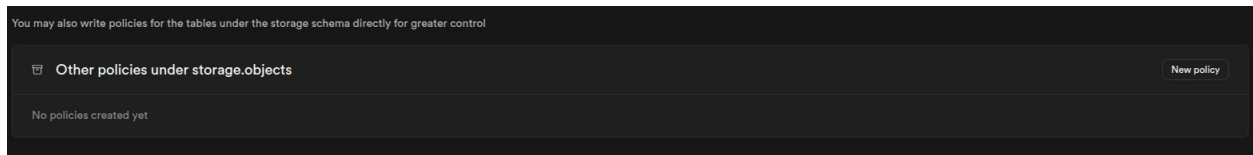
game-saves



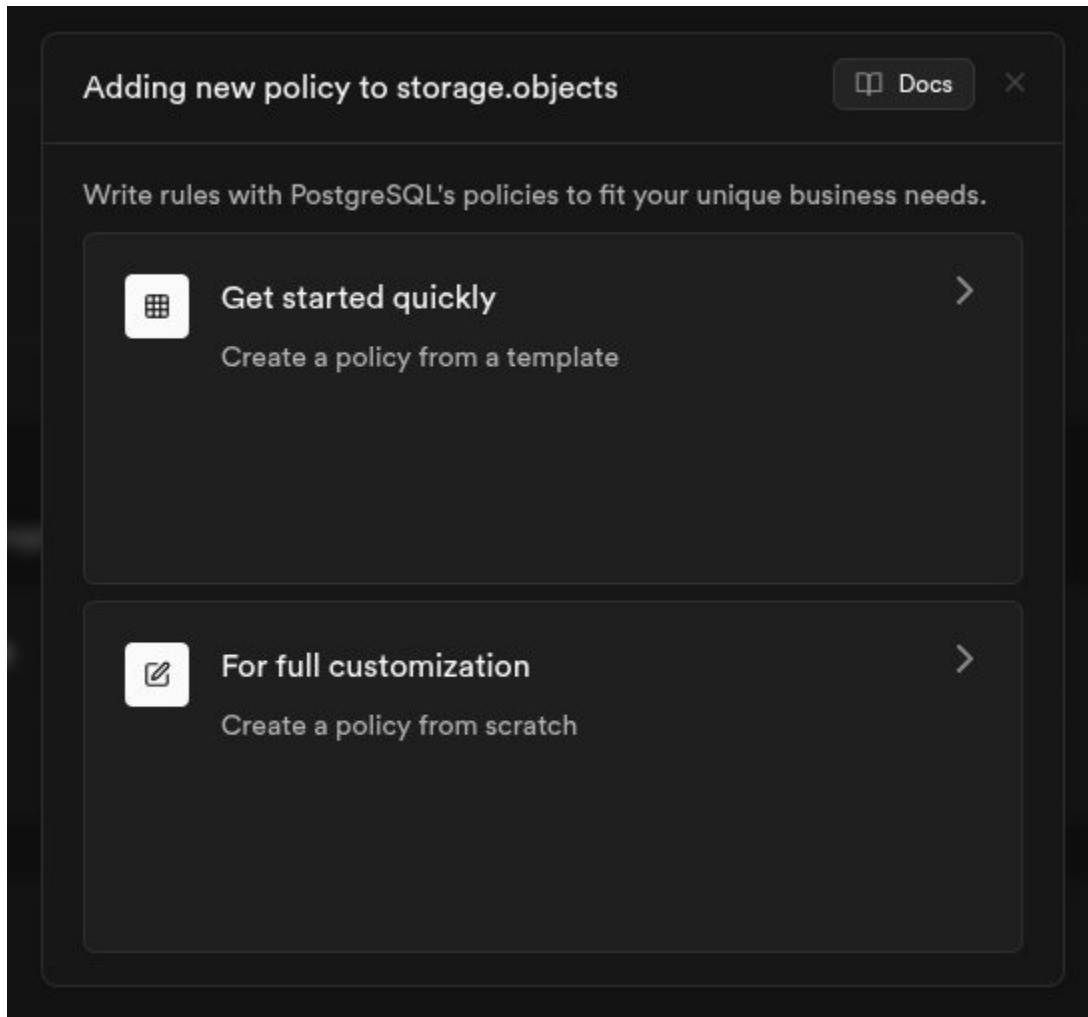
5. Click on Policies



6. Click on New Policy in the tab 'Other policies under storage.objects'



7. A new window opens. Click on 'For full customization'



8. Create the SELECT Policy

- **Policy Name:** Anon read game-saves
- Allowed Operation: SELECT
- Target roles: anon
- USING expression: `bucket_id = 'game-saves'`

Adding new policy to storage.objects

Docs

Policy name

A descriptive name for your policy

Anon read game-saves20/63

Allowed operation

Select an operation for this policy

SELECT

INSERT

UPDATE

DELETE

ALL

Target roles

Apply policy to the selected roles

anon

USING expression ⓘ

Provide a SQL conditional expression that returns a boolean.

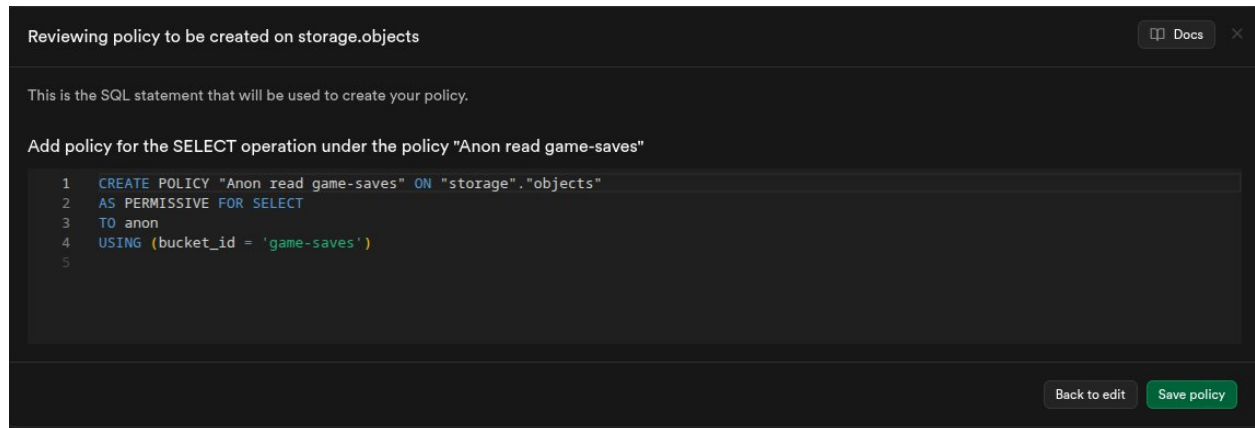
1

bucket_id = 'game-saves'

View templates

Review

9. Click on the green Review button and you will see this:



- Click on the **Save policy** button

10. Do the same for the other policies INSERT, UPDATE, and DELETE

```
-- INSERT or WRITE (upload)

CREATE POLICY "Anon insert game-saves"

ON storage.objects

FOR INSERT

TO anon

WITH CHECK (bucket_id = 'game-saves');
```

```
-- OVERWRITE or UPDATE (upsert)

CREATE POLICY "Anon update game-saves"

ON storage.objects

FOR UPDATE

TO anon

USING (bucket_id = 'game-saves')

WITH CHECK (bucket_id = 'game-saves');
```

```
-- DELETE (optional: player can wipe a slot)

CREATE POLICY "Anon delete game-saves"

ON storage.objects

FOR DELETE

TO anon

USING (bucket_id = 'game-saves');
```


11. Setup Crystal Save: Open Crystal Save Settings
12. Enable Cloud Save
13. Set Save Backend to Supabase
14. Enter your Supabase Project URL and Anon Key aka API key (You find them at the bottom of the project Dashboard)
15. Enter your the Bucket Name. We used **game-saves** in this example.
16. Pick a Supabase User ID Strategy (read further below)
17. Upon Saving, Crystal Save will generate a new folder named 'users' under 'game-saves' which contains a subfolder named 'guest'. In this folder will be the save files.

18. Supabase User ID Strategy:

Supabase Storage – Row-Level Security (RLS) Cheatsheet

Strategy (SaveSettings.supabaseUserIdStrategy)	Minimum RLS rule on storage.objects	Why / What it does
Shared (default)	sql bucket_id = 'game-saves' 	• One global “guest” folder (users/guest/...). • Every anon key can read & write every file in this bucket.
Public Per Build		• Public Per Build relies on the <i>bucket</i> (or <i>anon key</i>) itself being different per release, so the same rule is fine.
Guid Per Device	<i>(easiest)</i> make the bucket public <i>(tighter)</i> reuse the Shared rule above	• The client invents a random GUID once and keeps it in PlayerPrefs. • The server doesn't know that GUID – nothing inside auth.uid() to compare against – so it can't isolate users. • In practice this is no more or less secure than a public bucket, but keeps files grouped under users/{guid}/.
Unity Authentication	```sql bucket_id = 'game-saves' AND name LIKE ('users/'	
Custom	You decide — depends on what your IUserFolderResolver returns. Two options: 1. Public bucket or the simple bucket_id = 'game-saves' rule if you don't need server-side checks. 2. A bespoke expression that matches your resolver's pattern (e.g. `name LIKE ('org/'	

Supabase RLS (Row-Level Security) and the Custom–JWT strategy

When you switch Crystal Save's **Supabase User-Folder Strategy** to **Custom** and supply a per-player JWT through an `ISupabaseAuthorizationResolver`, two things matter:

1. **Which database role the token carries** – by default this is either anon or authenticated.
2. **What folder path your resolver returns** – for example `users/<uid>/...` or a company-wide scheme such as `org/<orgId>/users/<uid>/...`

As long as the role and the path still match your existing storage policies you do **not** need to change anything.

You *do* need to touch RLS when you issue tokens with a different role *or* change the path pattern.

Minimal per-player private-folder policies

The four policies on the next page allow the chosen role to read, write, overwrite and (optionally) delete files that live **only inside its own user folder**.

Adapt the snippets if any of these change:

- bucket name (game-saves)
- player folder prefix (users/)
- role (anon or authenticated)

-- 1. READ (SELECT)

```
create policy "Player read own saves"
on storage.objects
for select
to authenticated          -- or anon
using (
  bucket_id = 'game-saves'      -- your bucket
  and name like ('users/' || auth.uid() || '/%')
);
```

-- 2. INSERT (new upload)

```
create policy "Player insert own saves"
on storage.objects
for insert
to authenticated
with check (
  bucket_id = 'game-saves'
  and name like ('users/' || auth.uid() || '/%')
);
```

-- 3. UPDATE (overwrite / upsert)

```
create policy "Player update own saves"
on storage.objects
for update
to authenticated
using (
  bucket_id = 'game-saves'
  and name like ('users/' || auth.uid() || '/%')
)
with check (
  bucket_id = 'game-saves'
  and name like ('users/' || auth.uid() || '/%')
);
```

-- 4. DELETE (optional – let players wipe slots)

```
create policy "Player delete own saves"
on storage.objects
for delete
to authenticated
using (
  bucket_id = 'game-saves'
  and name like ('users/' || auth.uid() || '/%')
);
```

How it works

- bucket_id = 'game-saves' restricts access to that bucket only.
- auth.uid() is injected by Supabase from the JWT's sub claim.
- The name like ('users/' || auth.uid() || '/') predicate limits CRUD actions to the caller's own folder.
- with check (...) is required on INSERT and UPDATE to ensure the new path still follows the rule.

Issuing the player-scoped JWT

1. **Authenticate** the user in your back-end or via Supabase Auth.
2. **Generate** a JWT with the desired role (authenticated, anon, or a custom Postgres role).
3. **Give** that token to your game client
4. **Store** it inside your resolver:

```
46
47 using Arawn.CrystalSave.Runtime;
48 using UnityEngine;
49
50 [CreateAssetMenu(menuName = "Crystal Save/Custom Folder & Auth")]
51 public class MySupabaseResolver : ScriptableObject,
52     ISupabaseAuthorizationResolver
53 {
54     [SerializeField] string userId; // "d115f253-..."
55     [SerializeField] string jwtToken; // user-scoped JWT
56
57     public string ResolveUserFolder() => $"users/{userId}";
58     public string ResolveAccessKey() => jwtToken; // Crystal Save will use this as apikey + Bearer
59 }
60
```

FAQ

- **Do I also need policies on `storage.buckets`?**

Only if your game will create new buckets at runtime. Most projects keep a single bucket game-saves and protect **`storage.objects`**.

- **What about the “Guid Per Device” security story?**

GUIDs prevent players overwriting each other’s slots, but anyone who learns another GUID could fetch those files. Use this when you need per-device separation but don’t require strong auth.

- **Can I combine strategies?**

Yes. Example: set the enum to **`UnityAuthentication`** and in code fall back to **`Shared`** when `AuthenticationService.Instance.IsSignedIn == false`.

- **How do I secure the *Custom* strategy?**

The server has no built-in knowledge of your folder scheme, so you must write an RLS policy that matches the pattern your resolver returns (or leave the bucket public if that’s acceptable).

- **How do I give each player a private JWT?**

Issue the token from your backend (or Supabase Edge Function) after login and pass it into `MySupabaseResolver.jwtToken`. Crystal Save will use it automatically.